BookMyShow-Scale Ticketing Backend

Database Design, SQL Solution, Concurrency Strategy and Load-Test Plan

1. Scope

The solution models theatres, screens, seats, movies, shows, bookings, temporary holds and payments using MySQL. Redis is proposed for high-contention temporary seat locks with TTL. MySQL remains the persistent source of truth.

2. Tables and Purpose

Table	Purpose
users	Customers
theatres	Cinema locations
screens	Screens inside theatres
seats	Physical seats
movies	Movie catalogue
shows	Movie scheduled on a screen/date/time
show_seats	Per-show seat inventory
bookings	Booking/order
booking_seats	Seats in a booking
seat_holds	Temporary holds
payments	Payment transactions

3. Key Schema Rules

• Each theatre can have many screens; each screen can have many physical seats.

• A show connects one movie to one screen on a particular date and start/end time.

• show_seats materializes seat inventory per show, so the same physical seat can be available independently for different showtimes.

• UNIQUE(show_id, seat_id) prevents duplicate seat inventory for one show.

• Composite foreign keys ensure booking seats and holds reference a seat belonging to the same show.

• gateway_payment_id is UNIQUE to make payment webhooks idempotent.

4. Normalization

1NF: Attributes are atomic; seats are represented as rows rather than comma-separated values.

2NF: Relationship-specific data is kept in junction tables such as booking_seats rather than partially depending on a composite relationship.

3NF: Theatre, screen, seat, movie and payment details are separated so non-key attributes do not depend on other non-key attributes.

BCNF: Functional determinants are represented by primary/candidate keys and unique constraints, including show-seat identity.

5. Concurrency and Seat Locking

Thousands of users may request the same seat simultaneously. Redis handles the short-lived high-contention lock using an atomic SET NX operation with a TTL. Only the first request obtains the lock. A five-minute TTL releases abandoned holds automatically. MySQL constraints and transactions protect the permanent booking state.

```
SET seat-lock:{show_id}:{seat_id} {hold_token} NX EX 300

SUCCESS -> seat temporarily held
FAILURE -> another request already holds the seat
TTL expires -> temporary lock is released
```

6. Payment Idempotency

Payment gateways may retry webhooks. The payments table has a UNIQUE gateway_payment_id. A webhook with an already-processed gateway ID is treated as a duplicate instead of creating another payment transaction.

7. P2 Query

```
SELECT
    t.name AS theatre_name,
    m.title AS movie_name,
    sc.screen_name,
    s.show_date,
    s.start_time,
    s.end_time
FROM shows s
JOIN movies m ON s.movie_id = m.movie_id
JOIN screens sc ON s.screen_id = sc.screen_id
JOIN theatres t ON sc.theatre_id = t.theatre_id
WHERE t.theatre_id = 1
  AND s.show_date = '2026-10-01'
ORDER BY s.start_time;
```

8. Load-Test Plan

Test	Workload	Expected invariant
Same seat	100 concurrent users, same show + seat	Exactly one Redis lock succeeds; others fail
Multiple seats	100 users competing for 10 seats	At most 10 active holds
Webhook retry	100 identical payment webhooks	One payment record for the gateway ID
Hold expiry	Hold with 300-second TTL	Seat becomes available after expiry if payment did not comp

9. Submission Files

schema.sql contains the executable MySQL DDL. sample_data.sql contains example rows. queries.sql contains P2 and integrity checks. README.md explains the architecture and execution steps.